CASE-STUDY PRACTICE | CAPSTONE 14 | STUDY EDITION

Computer-Use Agents: Screenshots In, Coordinates Out, and the Cost of Avoiding an API

Scenario questions on an agent that completes a task by operating a graphical interface, perceiving screenshots and emitting mouse and keyboard actions, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on a computer-use system: figures are the sources' own claims or arithmetic from stated inputs, and token counts, latencies, costs and grounding accuracies are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the objective and reading the benchmark 6

2 Sizing the loop and the cost of avoiding an API 10

3 Perception, grounding, and the agent core 14

4 Serving, security, and operating the agent 18

Sources and confidence 22

Every question, by part

Part 1 Framing the objective and reading the benchmark 6

01 GUI or direct API? Framing 6

02 Three bars of success Evaluation 7

03 Read the OSWorld score Evaluation 8

04 Do not conflate the two scores Evaluation 9

Part 2 Sizing the loop and the cost of avoiding an API 10

05 Cost one screenshot task Sizing 10

06 Time the latency chain Sizing 11

07 Compare to a direct API Sizing 12

08 Sensitivity and what cannot be cached Sizing 13

Part 3 Perception, grounding, and the agent core 14

09 What does the agent perceive? Data 14

10 Review the click handler Code review 15

11 Define action grounding Agent design 16

12 Why observe every step? Agent design 17

Part 4 Serving, security, and operating the agent 18

13 Why not one synchronous request? Serving 18

14 Why is prompt injection acute here? Security 19

15 Monitor the grounding failure mode Operations 20

16 The cautious-rollout tension Operations 21

The case in one page

A summary of the case study to read before the questions.

Problem	Complete a natural-language task by operating a piece of software's own graphical interface, looking at a screenshot and emitting mouse and keyboard actions, the way a human would.
The approach	One general capability, not one integration per app. Anthropic: 'we're teaching it general computer skills' (source-confirmed). Ten low-level GUI primitives as the whole toolset.
Central trade-off	Screenshots in, coordinates out, costs about 64 times a direct API and tens of times the latency. Chosen only when no usable API exists, never as a default.
Benchmark	OSWorld screenshot-only: 14.9%, next-best 7.8%, 22.0% with more steps; SWE-bench Verified 33.4% to 49.0% (self-reported). OSWorld's own humans score about 72%.
Not published	Anthropic's latency, per-action grounding accuracy and real costs. Every token, latency and cost figure here is this document's.
First to break	Action grounding (a right plan, a click that misses) and prompt injection (the whole screen is the input, so on-screen text can hijack the agent).
Easy-to-miss traps	A pixel is not a semantic target. A benchmark score is not real completion. The screenshot cannot be cached. One moderate task already outlasts a 60-second timeout.

Numbers to know

Each figure carries the tag that says how it was obtained. Anthropic's OSWorld and SWE-bench figures are its own launch claims. Token, latency and cost figures are illustrative, because the announcement publishes none.

Quantity	Value	Evidence
OSWorld, Anthropic's claim	14.9% screenshot-only; 7.8% next-best; 22.0% with more steps	source-confirmed (Anthropic, self-reported)
OSWorld baselines	369 tasks on Ubuntu; humans about 72% (72.36%); best 2024 model about 12% (12.24%)	source-confirmed (OSWorld paper)
SWE-bench Verified	33.4% to 49.0% with computer use; a model-reasoning result, not this loop	source-confirmed (Anthropic, self-reported)
Resolution	XGA 1024x768 (4:3); WXGA 1280x800; FWXGA 1366x768; sizes above XGA/WXGA not recommended	source-confirmed (computer.py)
Action vocabulary	Action_20241022: 10 primitives (key, type, mouse_move, left_click, left_click_drag, right_click, middle_click, double_click, screenshot, cursor_position)	source-confirmed (computer.py)
Settle delay	reference implementation _screenshot_delay = 2.0 seconds before the next screenshot	source-confirmed (computer.py)
Tokens per screenshot	roughly 1,000 to 1,800 input tokens; midpoint 1,400 used here	source-confirmed (current API docs; dated separately)
Token cost per task	24 turns; (1,400 + 800) input, 150 output; 0.0528 x 3 + 0.0036 x 15 = about 0.21 dollars	illustrative; recomputed
Latency per step	100 + 150 + 2,000 + 50 + 2,000 = 4,300 ms; 24 steps = about 103 seconds	illustrative (inference) and source-confirmed (settle); recomputed
Direct-API comparison	2 calls at 300/50 tokens = 0.0033 dollars; cost about 64 to 1; latency about 50 to 100 to 1	illustrative; recomputed
Sensitivity	inference at 4,000 ms gives 6,300 ms a step and about 151 seconds a task	illustrative; recomputed
Timeout	about 103 s task against a 60 s ALB idle timeout; exceeds it with one moderate task	source-confirmed (ALB default); recomputed
Compounding error	0.95 to the 24th power is about 0.292, a 29.2% clean-run chance with no correction	illustrative; recomputed

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Interface	Operate the GUI with screenshots and mouse/keyboard	A direct API wherever one covers the task	source-confirmed; reasoned
Action space	Ten general GUI primitives as the whole toolset	Per-app semantic functions where an API exists	source-confirmed
Screen representation	One flat screenshot	A structured accessibility tree where the app exposes one	source-confirmed; reasoned
Coordinate frame	Map the scaled frame back to physical before clicking	Skip mapping only if no scaling applies	source-confirmed; reasoned
Control-flow pattern	ReAct: observe a fresh screenshot after each action	Blind execution of a pre-planned action list	source-confirmed; reasoned
Serving	Fast submit, task queue, progress channel	One synchronous request for a sub-timeout task	source-confirmed; reasoned
Isolation	A dedicated sandboxed display per task	A shared desktop if the blast radius is acceptable	reasoned
Hard elements	Prefer a keyboard shortcut over a mouse coordinate	A mouse click for large, unambiguous targets	source-confirmed
Rollout	Gate early use to low-risk, reversible tasks	Broad use where a wrong action is cheap	source-confirmed; reasoned

PART 1

Framing the objective and reading the benchmark

Why this agent operates a GUI rather than an API, what the three bars of success are, and how to read a low benchmark score and a coding score that does not belong to this loop.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Computer-use agent	An agent whose only way of acting on a target system is through its graphical interface, perceived as a screenshot and acted on with emulated mouse and keyboard input.
GUI	Graphical user interface: the visual, screen-based interface a human operates with a mouse and keyboard, as opposed to a programmatic interface an API exposes.
General computer skills	Anthropic's stated design: one general capability that uses a small set of mouse and keyboard primitives across any software, rather than one bespoke tool per task.
Three bars of success	Real task completion (the external system ends correct), benchmark grading (an automated check of a test environment's final state), and per-action grounding (each click did what was intended).
OSWorld	A benchmark and execution environment of real, open-ended computer tasks across real applications, graded by a script that checks the environment's actual final state, not the transcript.
Screenshot-only category	An OSWorld setting where the agent perceives only screenshots, with no structured accessibility data, matching this architecture exactly.
Execution-based grading	Scoring a task by running an automated check against the real resulting state of the environment, rather than by a human or model judging whether the run looks right.
Self-reported figure	A number published by the party that benefits from it, often at launch. It is evidence, not an independent audit.
Where to read	Anthropic: Developing a computer use model (October 22, 2024) | Anthropic computer-use reference implementation, computer.py (anthropic-quickstarts) | Xie et al.: OSWorld, Benchmarking Multimodal Agents in Real Computer Environments (NeurIPS 2024, arXiv:2404.07972)

01 GUI or direct API?

FRAMING

SCENARIO

A product manager says the agent should just call each app's API, so why build something that stares at a screen and moves a mouse. Explain the objective and why operating the GUI is the design, not an oversight.

HOW TO THINK ABOUT IT

State what the agent perceives and emits, and what 'general computer skills' buys.

Q What is the objective?

A Given a task in natural language and a piece of software, complete it by operating the software's own graphical interface, the same one a human uses: look at a screenshot, move a cursor, click and type (source-confirmed).

- The input is a screen, the output is mouse and keyboard actions.

Q What is 'general computer skills'?

A Anthropic's stated choice: 'Instead of making specific tools to help Claude complete individual tasks, we're teaching it general computer skills' (source-confirmed). One capability reusable across any visual software, not one integration per app.

- A small set of GUI primitives, not a rich per-app function library.

Q Why not just call the API?

A Often there is no usable API (Part 2 covers this). Operating the GUI is the only option for software that exposes no programmatic interface, which is most installed software.

- The GUI is the fallback that always exists, because a human can always use it.

Q Is this the better choice in general?

A No. When a usable API exists it wins on cost and latency (Part 2). The GUI loop is chosen only when no API does, which is the point of the whole trade-off.

- General, but slower and more error-prone; a last resort, not a default.

Q Reframe it.

A Ask whether a programmatic interface exists before reaching for the screen. The GUI loop earns its cost only where no API covers the task.

Basis: The objective and the 'general computer skills' quote are Anthropic's own (source-confirmed); the 'last resort' framing is this document's (reasoned).

02 Three bars of success

EVALUATION

SCENARIO

An engineer wants one pass or fail number for the agent. Explain the three different bars and why per-action success is not the same as task success.

HOW TO THINK ABOUT IT

Order the bars from loosest to strictest and say how they relate.

Q What is the real-world bar?

A The external system ends in the right state: a field holds the correct value, a file was created, a booking was made. The only bar an end user cares about, and the hardest to check automatically.

- It needs a scripted check against the real system or human inspection.

Q What is the benchmark bar?

A An automated, execution-based check against a test environment's actual final state (OSWorld, Part 4), not a human or model judging the transcript. Reproducible, but a narrower proxy.

- Cheaper and repeatable, but a fixed task set, not real arbitrary software.

Q What is per-action grounding?

A Whether each individual click, keystroke or scroll did what the model intended. A necessary condition for task success, not a replacement for it.

- Defined in full in Part 3; it is this architecture's own failure mode.

Q Why is grounding not the same as task success?

A Every action can be correctly grounded and the task still fail, from a wrong plan or a misread instruction. Correct execution of each step does not mean the right steps were chosen.

- The same split Lesson 1 draws between acting correctly and deciding the right action.

Q Reframe it.

A Name which bar a number measures. A high per-action grounding rate does not prove tasks complete, and a benchmark score does not prove real software ends up correct.

Basis: The three bars are this document's framing built on the sources (reasoned); the execution-based grading is OSWorld's own (source-confirmed).

03 Read the OSWorld score

EVALUATION

SCENARIO

A slide quotes: Claude 3.5 Sonnet scored 14.9% on OSWorld screenshot-only, against 7.8% for the next best, and 22.0% with more steps. A reader takes this as near-solved. Say what the number means.

HOW TO THINK ABOUT IT

Define OSWorld and its grading, then read the figure against the benchmark's own human baseline.

Q What does OSWorld grade?

A An agent's ability to complete real, open-ended computer tasks across real applications (369 tasks on Ubuntu), checked by a script against the environment's actual final state. The screenshot-only category matches this architecture (source-confirmed).

- Real applications, execution-based grading, no structured accessibility data.

Q What are the confirmed figures?

A 14.9% screenshot-only, 7.8% next-best, and 22.0% with more steps, all quoted from Anthropic's announcement (source-confirmed, self-reported).

- Anthropic reports them; they are a launch claim.

Q How low is that, really?

A Low in absolute terms. OSWorld's own paper reports humans completing about 72% of the task set against the best 2024 model near 12%. Even the headline 14.9% resolves well under a quarter of real tasks (source-confirmed, OSWorld paper).

- The human baseline is the yardstick, not the next-best model.

Q What does the gap to 22.0% show?

A More steps help, which fits a loop that catches and corrects errors over turns (Part 3), but the ceiling is still far below human. It is 'experimental... cumbersome and error-prone' in Anthropic's own words.

- Improvement with steps is real; the absolute level is still low.

Q Reframe it.

A Read a benchmark score against the benchmark's own human baseline, not against the next-best system. A lead over rivals can still be far from usable.

Basis: The figures and caveat are Anthropic's own (source-confirmed, self-reported); the human and best-model baselines are from the OSWorld paper (source-confirmed).

04 Do not conflate the two scores

EVALUATION

SCENARIO

The same announcement says computer use 'improves performance on SWE-bench Verified from 33.4% to 49.0%'. A teammate cites this as proof the screenshot loop works well. Explain why that is the wrong reading.

HOW TO THINK ABOUT IT

Ask what each benchmark measures and whether it is this capstone's loop.

Q What does SWE-bench Verified measure?

A The underlying model's general reasoning and coding ability on GitHub-issue tasks, the benchmark family capstone #13 defines. It is a coding result, graded by tests (source-confirmed).

- It is about the model's code reasoning, not about operating a screen.

Q What does OSWorld measure?

A This capstone's own screenshot-and-action loop: perceiving a screen and emitting mouse and keyboard actions.

- Different input, different output, different failure mode.

Q Why not conflate them?

A They were announced together but measure different things. A jump on SWE-bench Verified says the model got better at coding, not that the computer-use loop is reliable, which OSWorld's 14.9% actually speaks to.

- Announced together is not the same as measuring the same capability.

Q What should you cite for the loop?

A The OSWorld screenshot-only figure, read against the human baseline, plus Anthropic's own 'imperfect' and 'low-risk tasks' caveats.

- Cite the benchmark that actually exercises the architecture in question.

Q Reframe it.

A Match each metric to the capability it exercises. A strong number on an unrelated benchmark is not evidence for the system you are actually asking about.

Basis: Both figures are Anthropic's own (source-confirmed); which capability each measures is reasoned from the benchmarks' definitions.

PART 2

Sizing the loop and the cost of avoiding an API

Costing one screenshot task, timing its latency chain, comparing it to a direct API, and naming the term the estimate depends on and the one that cannot be cached.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Back-of-envelope method	State assumptions, decompose, compute carrying units, sanity-check the order of magnitude, and name the input the answer is most sensitive to.
Round-trip	One turn of the loop: capture a screenshot, send it with the history, get one action back, execute it, wait to settle.
Token pricing anchor	This repo's shared illustrative rate: 3 dollars per million input tokens and 15 dollars per million output tokens. For consistency, not a real provider price.
Settle delay	A deliberate pause after an action before the next screenshot, so the screen finishes updating. The reference implementation sets 2.0 seconds.
Direct-API alternative	Doing the same logical task through a programmatic interface: a couple of structured calls, no images, far cheaper and faster.
Sensitivity	How much the answer moves when one assumption changes. The most sensitive input is the one to measure first.
Prompt caching	Serving a stable prompt prefix from cache so it is not recomputed each turn. New, unique content such as a fresh screenshot cannot be cached.
Where to read	Anthropic: Developing a computer use model (October 22, 2024) | Anthropic computer-use reference implementation, computer.py (anthropic-quickstarts) | Xie et al.: OSWorld, Benchmarking Multimodal Agents in Real Computer Environments (NeurIPS 2024, arXiv:2404.07972)

05 Cost one screenshot task

SIZING

SCENARIO

A moderate task takes 24 round-trips, each sending a 1,400-token screenshot plus 800 tokens of history and returning 150 output tokens, at 3 dollars per million input and 15 per million output. Estimate the token cost.

HOW TO THINK ABOUT IT

Find the input and output totals across round-trips, then price them separately.

Q What is the input per round-trip?

A 1,400 screenshot tokens plus 800 history tokens is 2,200 input tokens a turn (recomputed).

- The screenshot is most of it.

Q What are the totals?

A 24 turns times 2,200 is 52,800 input tokens, 0.0528 million. Times 150 output is 3,600 tokens, 0.0036 million.

- Keep input and output separate; they are priced differently.

Q What is the cost?

A 0.0528 times 3 is 0.1584 dollars of input, plus 0.0036 times 15 is 0.054 dollars of output, about 0.21 dollars per task (recomputed).

- The screenshot-laden input dominates the bill.

Q Is this a floor or a ceiling?

A A floor for a short task. Anthropic's docs recommend pruning old screenshots because they accumulate, so a long task costs more per turn as history grows.

- Flat per-turn tokens understate a long run.

Q Reframe it.

A Price the dominant term first. Here the per-turn screenshot, not the output, sets the cost, and it grows with task length.

Basis: The token counts and prices are illustrative (screenshot range from Anthropic's docs); the cost is arithmetic (recomputed).

06 Time the latency chain

SIZING

SCENARIO

Each round-trip is screenshot capture 100 ms, encode and upload 150 ms, model inference 2,000 ms, action execution 50 ms, and a settle delay of 2.0 seconds. Estimate per-step and total latency for a 24-step task.

HOW TO THINK ABOUT IT

Sum the per-step terms, then multiply by the step count, and compare to a known timeout.

Q What is per-step latency?

A 100 plus 150 plus 2,000 plus 50 plus 2,000 is 4,300 ms, about 4.3 seconds a step (recomputed).

- Model inference and the settle delay are the two large terms.

Q Where does the settle delay come from?

A The reference implementation sets a 2.0-second delay before the next screenshot, so the screen finishes updating (source-confirmed, computer.py). It is a real constant, not a guess.

- Confirmed from code, unlike the illustrative inference time.

Q What is the task total?

A 24 steps times 4.3 seconds is about 103 seconds.

- Linear in the step count.

Q Why does that matter for serving?

A It already exceeds AWS ALB's 60-second default idle timeout (Lesson 6). One moderate task cannot be a single synchronous request (Part 4).

- A short task crossing the timeout decides the serving shape.

Q Reframe it.

A Add the per-step terms, scale by steps, and compare the total to the timeouts your infrastructure actually enforces.

Basis: The settle delay is source-confirmed (computer.py); other terms are illustrative; the totals are arithmetic (recomputed).

07 Compare to a direct API

SIZING

SCENARIO

The same logical task through an API might take 2 structured calls at 300 input and 50 output tokens each, finishing in 1 to 2 seconds. Give the cost and latency ratios, then say when the GUI loop is still the right choice.

HOW TO THINK ABOUT IT

Compute both ratios, then name the cases where no usable API exists.

Q What is the direct-API cost?

A 2 calls: 600 input tokens and 100 output. 0.0006 times 3 plus 0.0001 times 15 is 0.0033 dollars (recomputed).

- No image, so the per-call tokens are tiny.

Q What are the ratios?

A Cost 0.2124 over 0.0033 is about 64 to 1; round-trips 24 over 2 is 12 to 1; latency about 103 seconds over 1 to 2 is roughly 50 to 100 to 1.

- Two independent ratios landing in the same band is a sanity check.

Q When is the loop still right?

A When no usable API exists: legacy software with no programmatic interface; an API that does not expose the needed toggle or export; or a task over arbitrary, not-known-in-advance software such as any website (source-confirmed design goal).

- The GUI is the interface that always exists because a human can use it.

Q What does this say about defaults?

A Reach for the API whenever one covers the task. The loop's generality is paid for at roughly 64 times the cost and tens of times the latency, so it is a last resort.

- Generality is expensive; use it only where nothing cheaper reaches.

Q Reframe it.

A Quantify the premium the GUI loop charges, then justify paying it only by the absence of a usable API, not by convenience.

Basis: The direct-API inputs are illustrative; the ratios are arithmetic (recomputed); the 'no API exists' cases are reasoned from Anthropic's stated goal.

08 Sensitivity and what cannot be cached

SIZING

SCENARIO

Someone proposes cutting cost by caching the prompt, and separately asks which assumption to measure first. Say which input the estimate is most sensitive to, and what caching can and cannot recover.

HOW TO THINK ABOUT IT

Name the least-grounded term, then split the per-turn content into cacheable and not.

Q Which input is most sensitive?

A Model inference time per round-trip. It is illustrative, since the 2024 announcement publishes no latency, while the settle delay is now a confirmed constant.

- The unmeasured term, not the confirmed one, is where the risk is.

Q What if inference doubles?

A At 4,000 ms, per-step latency rises to 6,300 ms and the 24-step task to about 151 seconds (recomputed), well past the 60-second timeout rather than just over it.

- Plausible once the action history has grown over many turns.

Q What is cacheable?

A The system prompt and the action-space definition, identical every turn, plus most of the 800 history tokens, which repeat with one new increment each turn.

- Stable prefixes are the cache's natural target.

Q What is not cacheable?

A The screenshot: 1,400 new, unique tokens every turn, the dominant cost term. It cannot be served from cache the way a repeated prompt can.

- Caching cannot claw back the cost that is new each turn.

Q Reframe it.

A Measure the least-grounded term first, and expect caching to help only the repeated prefix, not the fresh per-turn screenshot that drives the bill.

Basis: The sensitivity and caching split are this document's analysis (reasoned); the recomputed latency is arithmetic (recomputed).

PART 3

Perception, grounding, and the agent core

What the agent actually sees, the coordinate mapping that must be right, the grounding failure mode a tool call never has, and why observing each step is load-bearing.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Raw screenshot perception	The agent's input each step is a bitmap of the screen, not parsed elements. It must derive every button, label and value from pixels, as a human would.
Scaled coordinate frame	A resized frame (e.g. XGA 1024x768) the screenshot is sent in and the model emits coordinates in; it differs from the physical display whenever scaling applies.
Coordinate mapping	Translating a coordinate from the scaled frame the model used back to the physical screen before executing the click. If the mapping is off, every click misses by a fixed factor.
Action grounding	Translating an intended action (click the Submit button) into the correct low-level input (the exact pixel of that button on the current screen) so executing it has the intended effect.
Action-grounding error	The plan was right but the emitted coordinate missed the intended element, so the action registers mechanically while still being wrong.
ReAct	Reason, act, observe, repeat. This loop is a special case where the observation is always a raw screenshot and the action space is one fixed set of GUI primitives.
Compounding error	If each of n actions is correct with probability p, the chance all n are right is p to the power n, which falls fast as n grows.
Where to read	Anthropic: Developing a computer use model (October 22, 2024) | Anthropic computer-use reference implementation, computer.py (anthropic-quickstarts) | Xie et al.: OSWorld, Benchmarking Multimodal Agents in Real Computer Environments (NeurIPS 2024, arXiv:2404.07972)

09 What does the agent perceive?

DATA

SCENARIO

A colleague assumes the agent receives the page's buttons and fields as structured data, like a normal tool call. Explain what it actually perceives and why that forces a new concept.

HOW TO THINK ABOUT IT

Contrast this agent's input with a coding agent's and a tool call's, then name what pixels do not give you.

Q What is the input each step?

A One raw screenshot, a bitmap of whatever is on the target screen, plus the running history of its own actions and the original task (source-confirmed design).

- A picture, not parsed elements.

Q How does that differ from other agents?

A A coding agent perceives structured text, file contents and test output, already machine-readable. A support agent perceives structured records. This agent perceives pixels only.

- No button name, field label or value arrives pre-parsed.

Q What does it have to do with those pixels?

A Derive every element's identity and location itself, the way a human looking at the screen would, before it can act.

- Perception is an interpretation step, not a given.

Q Why does that force a new concept?

A Because the model does not name what it clicks, it names where, a pixel coordinate. Whether that coordinate lands on the intended element is a separate thing that can go wrong: action grounding (Q11).

- A JSON tool call has no coordinate to miss; a GUI click does.

Q Reframe it.

A When the input is a rendered screen, perception and grounding become real steps that can fail on their own, distinct from whether the plan was right.

Basis: The screenshot-only input is Anthropic's design (source-confirmed); the contrast with other agents is this document's framing (reasoned).

10 Review the click handler

CODE REVIEW

SCENARIO

On most screens the agent clicks fine, but on high-DPI laptops every click lands up and to the left of the target, by a consistent amount. The handler below executes the model's coordinate. Find the fault and fix it.

```
def do_click(model_coord, display):
    # model_coord came back from the model, in the scaled frame (e.g. XGA 1024x768)
    x, y = model_coord
    display.click(x, y)   # clicked straight onto the physical screen
```

HOW TO THINK ABOUT IT

Ask which coordinate frame the model emits in, and which frame the display expects.

Q Which frame does the model use?

A The scaled frame the screenshot was sent in, for example XGA 1024x768, not the physical display's resolution (source-confirmed: the model emits coordinates in the scaled frame).

- The screenshot was downscaled before sending; the model answers in that size.

Q What does the handler do wrong?

A It clicks the model's scaled coordinate straight on the physical screen. On a high-DPI display the physical screen is larger in pixels, so every click lands short by the scale factor.

- Right area, consistent miss: the signature of a scaling error, not a model error.

Q What is the fix?

A Map the scaled coordinate to the physical one before clicking: multiply by the ratio of physical size to scaled size on each axis, then round. The correction must live in exactly one place.

- Equivalently, downscale the screenshot and upscale the coordinate consistently.

Q How would you catch it?

A Test on a 2x display with a known target, and check a corner click lands in the corner. A consistent fixed offset points at scaling, not at grounding.

- A fixed-factor miss is reproducible; verify by hand before blaming the model.

Q Reframe it.

A When a model emits coordinates in a scaled frame, map them back to the physical frame once, deterministically. A consistent offset is a units bug, not a reasoning bug.

REFERENCE CODE

```
def do_click(model_coord, display, scaled_wh, physical_wh):
    # the model answers in the scaled frame; map back to physical before clicking
    sx, sy = scaled_wh       # e.g. (1024, 768)
    px, py = physical_wh     # e.g. (2048, 1536) on a 2x display
    mx, my = model_coord
    x = round(mx * px / sx)  # one deterministic mapping, in one place
    y = round(my * py / sy)
    display.click(x, y)
```

Basis: The scaled-frame behaviour is source-confirmed (computer.py); the bug and fix are a standard coordinate-mapping error, reasoned and checked by hand, not run.

11 Define action grounding

AGENT DESIGN

SCENARIO

An interviewer asks what action grounding is, why it is specific to this architecture, and what makes some elements harder than others. Give the definition and a mitigation.

HOW TO THINK ABOUT IT

Contrast a coordinate action with a named tool call, then name the hard cases and a way around them.

Q What is action grounding?

A Translating an intended action (click the Submit button) into the correct low-level input (the exact pixel of that button on the current screen) so executing it produces the intended effect.

- Intent to concrete input, on the screen as it is now.

Q What is a grounding error?

A The plan was right but the emitted coordinate missed the intended element, so the click registers with no error while still being wrong.

- A mechanically successful action that does the wrong thing.

Q Why is it specific to this architecture?

A A named tool call like propose_refund(id, amount) carries the meaning in its arguments. A GUI action names only a coordinate, which is correct only if it lands on the intended element; intent and coordinate can diverge.

- A coordinate has no semantic identity to check against.

Q Which elements are hard, and what helps?

A Scrollbars and dropdowns: small, ambiguous targets Anthropic's docs flag as tricky by mouse. The mitigation sidesteps grounding: use a keyboard shortcut (arrow keys), which has no coordinate to miss (source-confirmed, dated separately).

- Change the action type to one with no coordinate rather than aim more carefully.

Q Reframe it.

A Grounding is the gap between a right intent and a right coordinate. Where a target is hard to hit, prefer an action with no coordinate at all.

Basis: The definition is this capstone's concept; the hard-element and keyboard-mitigation points are from Anthropic's docs (source-confirmed, dated separately).

12 Why observe every step?

AGENT DESIGN

SCENARIO

Someone proposes having the model plan all 24 actions up front and execute them without fresh screenshots, to save time. At 95% per-action grounding accuracy, estimate the clean-run chance and say what it implies.

HOW TO THINK ABOUT IT

Compute p to the power n for an uncorrected run, then interpret it.

Q What is the formula?

A With no correction between actions, the chance all n are correctly grounded is p to the power n, here 0.95 to the 24th power (reasoned model).

- Each action independent and unchecked is the assumption.

Q What is the value?

A 0.95 to the 24th power is about 0.292, roughly a 29.2% chance of a fully clean run (recomputed).

- Nearly 71% of the time at least one of 24 actions is mis-grounded.

Q Why does a fresh screenshot help?

A It is ReAct's observe step: the model sees the actual result and can catch a mis-grounded click before the next action builds on it. Anthropic's documented practice is a screenshot after essentially every action.

- Observation turns a blind sequence into one that can self-correct.

Q Is this Anthropic's real number?

A No. Anthropic publishes no per-action grounding accuracy; 95% is illustrative, shown only to make the compounding concrete, like capstone #13's own per-step figure.

- The shape of the argument is the point, not the exact value.

Q Reframe it.

A When success needs many actions all correct, p to the power n shows why blind execution fails and why observing each result is structural, not optional.

Basis: The 95% rate is illustrative; 0.95 to the 24th power is arithmetic (recomputed); the screenshot-after-every-action practice is from Anthropic's docs (source-confirmed, dated separately).

PART 4

Serving, security, and operating the agent

Why a short task still cannot be one request, why prompt injection is unusually acute here, monitoring the grounding failure mode, and the tension in a cautious rollout.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Idle timeout	The longest a gateway lets one request wait for a response before closing it. AWS ALB defaults to 60 seconds.
Task queue	A queue of background jobs with status and retries, run by a worker pool, with a separate channel for progress.
Indirect prompt injection	An instruction planted in third-party content the model reads as part of its job, such as a web page it is viewing, that hijacks its behaviour.
Blast radius	How much a single breach or bad action can reach. Here it equals whatever the actions can reach on the display, the same as a human logged into that session.
Action-grounding metric	A monitored rate: for sampled tasks, whether each action's intended target matched its actual on-screen effect, checked from the next screenshot.
Low-risk-tasks gate	Anthropic's recommendation to start with tasks where a wrong action is cheap to detect and reverse, before consequential ones.
Sampled audit	Reviewing a subset rather than every task; scales to volume but can miss a rare failure mode for longer.
Where to read	Anthropic: Developing a computer use model (October 22, 2024) | Anthropic computer-use reference implementation, computer.py (anthropic-quickstarts) | Xie et al.: OSWorld, Benchmarking Multimodal Agents in Real Computer Environments (NeurIPS 2024, arXiv:2404.07972)

13 Why not one synchronous request?

SERVING

SCENARIO

A colleague wants the submit call to hold the connection open until the task finishes. Given a 24-step task takes about 103 seconds, explain why that fails and what the shape should be.

HOW TO THINK ABOUT IT

Compare the task time to the gateway timeout, then give the decomposition.

Q What is the timeout problem?

A About 103 seconds already exceeds AWS ALB's 60-second default idle timeout (Lesson 6). The gateway closes the connection before the task finishes, even for a moderate task.

- Not a multi-hour edge case; a short task already crosses it.

Q What is the submit shape?

A A REST call that returns a task id immediately, well under the timeout, with the work running elsewhere.

- Accept fast, run out of band.

Q Where does the loop run?

A In a background job pulled from a task queue, running the screenshot-reasoning-action loop, decoupled from the submit request. The same shape capstone #13 built.

- One queue entry is one task; a worker owns the loop.

Q How does the caller follow it?

A A separate progress channel: poll a status endpoint or subscribe to a stream.

- Submit, queue, progress channel: three parts, reused not reinvented.

Q Reframe it.

A When even a moderate task outlives the gateway timeout, run it as a queued background job with a progress channel, not a held-open request.

Basis: The 103-second figure is this document's arithmetic (recomputed); the ALB timeout and the decomposition are from Lesson 6 (source-confirmed within the repo).

14 Why is prompt injection acute here?

SECURITY

SCENARIO

A reviewer treats prompt injection as a generic risk handled by a system prompt. Explain why this architecture is an unusually acute case and what bounds the damage.

HOW TO THINK ABOUT IT

Ask what the model's whole input is each step, then what the actions can reach.

Q What is the model's input each step?

A A rendered view of whatever is on screen. If that screen shows a malicious page, the injected instruction is the entirety of what the model perceives, not one document among many.

- The attack surface is the whole screen, every turn.

Q Why is a system prompt not enough?

A There is no separate trusted channel distinguishing on-screen content from instructions the way a labelled RAG chunk can. Anthropic's docs warn: 'Claude will follow commands found in content even when they conflict with your instructions' (source-confirmed, dated separately).

- On-screen text and the task arrive through the same visual channel.

Q What is the blast radius?

A Whatever the actions can reach on that display: the same reach a human operator logged into that session would have, since the agent acts exactly like one (Lesson 7).

- Acting like a human means the damage of a misled human.

Q What bounds it?

A Running the display in a dedicated, isolated sandbox per task (Lesson 8), so an injected action cannot reach other sessions or credentials, plus restricting early use to low-risk tasks.

- Isolation per task plus a cautious task scope.

Q Reframe it.

A When the whole input is attacker-reachable content with no trusted channel, assume injection will land and bound the blast radius with per-task isolation.

Basis: The injection warning is Anthropic's (source-confirmed, dated separately); the blast-radius and sandbox points follow Lessons 7 and 8 (reasoned).

15 Monitor the grounding failure mode

OPERATIONS

SCENARIO

A task fails OSWorld-style grading, and the team cannot tell whether the plan was wrong or a click missed. Design the monitoring that tells them apart, and where each failure routes.

HOW TO THINK ABOUT IT

Use data the loop already captures, then split failures by cause.

Q What does a failing grade not tell you?

A Why it failed: a wrong plan, or a right plan with a mis-grounded action. A single pass or fail blends the two.

- You need to separate deciding the action from executing it.

Q What should you log?

A For a sampled subset of tasks, whether each action's intended target, from the model's own stated reasoning, matched its actual on-screen effect, checkable from the very next screenshot the loop already captured.

- The next screenshot is free evidence of what the last action did.

Q What is that metric like?

A A rate at which a named failure mode is caught, like capstone #12's validator catch rate, here applied to action grounding. Break it out by UI-element type, since dropdowns and scrollbars are disclosed as harder.

- A blended number hides which element type drives the errors.

Q Where does each failure route?

A A grounding error routes to Step 3's resolution and scaling choices; a consistently misunderstood instruction routes to Step 1's task-instruction framing, a prompting problem, not a grounding one.

- Route by cause, the same feedback shape capstones #9, #12 and #13 use.

Q Reframe it.

A Instrument the specific failure mode, grounding, with the data you already capture, break it out by element type, and route each cause to the thing that fixes it.

Basis: The grounding metric and feedback loop are this document's design (reasoned); the harder-element disclosure is from Anthropic's docs (source-confirmed, dated separately).

16 The cautious-rollout tension

OPERATIONS

SCENARIO

A lead wants to deploy the agent broadly and audit every task. Explain the deployment gate Anthropic recommends, the audit trade-off, and the tension the safe path creates.

HOW TO THINK ABOUT IT

Weigh the low-risk gate and full-versus-sampled audit, then name what the safe path defers.

Q What gate does Anthropic recommend?

A Begin with low-risk tasks, where a mis-grounded action is cheap to detect and reverse, before consequential ones (source-confirmed: 'begin exploration with low-risk tasks').

- Match the task's reversibility to the architecture's error rate.

Q Full audit or sampled?

A Full review does not scale to real volume; sampling does but, like every sampled audit here (capstone #11's stratified audit, #12's validator-audit sample), can miss a rare UI pattern's failure for longer.

- Completeness against cost, the familiar trade-off.

Q What is the tension?

A The architecture's highest-value uses are irreversible, consequential actions on legacy software with no API, exactly Step 4's strongest justification for using it. The cautious rollout defers those longest.

- The safest path postpones the cases that most need the tool.

Q How do you resolve it?

A You do not resolve it in general; you decide per deployment by how costly a wrong action is there, and expand scope as the grounding metric and reversibility controls earn trust.

- A business decision the architecture does not make for you.

Q Reframe it.

A Gate by reversibility, sample audits by risk, and acknowledge that caution defers the highest-value, least-reversible tasks, then expand as measured reliability allows.

Basis: The low-risk gate is Anthropic's recommendation (source-confirmed); the audit trade-off and the tension are this document's analysis (reasoned).

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #14. The primary sources are Anthropic's 'Developing a computer use model' (October 22, 2024), read in full from the live page, Anthropic's own reference implementation computer.py, and the OSWorld paper (Xie et al., NeurIPS 2024), read for the benchmark's own definition. Anthropic's OSWorld figures (14.9% / 7.8% / 22.0%) and the SWE-bench Verified figures (33.4% to 49.0%) are its own launch claims. Anthropic's current API docs are cited only where flagged and dated separately from the 2024 announcement; the toolset has since evolved. The serving, grounding-monitoring and injection designs in Steps 5 and 6 build on this repo's lessons, not on a documented account of Anthropic's internals.

- Anthropic: Developing a computer use model, October 22, 2024, read in full from the live page 2026-10-17
https://www.anthropic.com/news/3-5-models-and-computer-use

- Anthropic computer-use reference implementation, computer.py (anthropic-quickstarts), read 2026-10-17
https://github.com/anthropics/anthropic-quickstarts

- Xie et al.: OSWorld, Benchmarking Multimodal Agents in Real Computer Environments, NeurIPS 2024 (arXiv:2404.07972), read 2026-10-17
https://arxiv.org/abs/2404.07972